PRÁCTICA OPENCODE UT2. ANALIZAR UNA ARQUITECTURA CON UN AGENTE (RESUELTA)

Antes de empezar

opencode instalado (ver la guía de instalación) y el proyecto en un repositorio Git, que es tu botón de deshacer. Comprueba con /models qué modelo tienes activo: con los modelos gratuitos tendrás que ser más explícito en los prompts y cometerán más errores, lo que viene bien para lo que se pide aquí. Necesitas además la aplicación tienda-libros de la carpeta recursos, copiada a tu carpeta de trabajo, y un visor de Markdown que dibuje diagramas Mermaid (Visual Studio Code con la extensión «Markdown Preview Mermaid Support», o el propio GitHub). No hace falta levantar la aplicación.

- Trabaja en modo Plan (tecla Tab) hasta que el plan te convenza; solo entonces pasa a Build.
- Revisa cada cambio antes de aceptarlo (git diff). Si no sabes explicar una línea, no la aceptes.
- Verifica ejecutando: la evidencia es la salida del comando, no la respuesta del agente.
- Anota cada prompt en prompts.md: qué pediste, qué hizo y qué corregiste.
- Nada de contraseñas ni datos personales en el chat: los modelos gratuitos son de prueba y no admiten información confidencial.
- Si el modelo se atasca, da vueltas o va muy lento, divide la petición en pasos o cambia de modelo con /models.

Ejercicios

Todo el trabajo se recoge en un documento docs/arquitectura.md dentro del proyecto, que escribes tú con la ayuda del agente. Todos los comandos de las respuestas se ejecutan dentro de la carpeta tienda-libros/.

1) Agente en solo lectura. Crea opencode.json con "edit": "deny", "bash": "ask" y "share": "disabled", arranca opencode en modo Plan y pide un resumen de la aplicación. Después pídele expresamente que modifique docker-compose.yml: captura el rechazo. Explica por qué en una fase de análisis conviene un agente que no pueda escribir. (1p)

Respuesta:

Creé opencode.json en el proyecto con la política de solo lectura:

```
$ cat opencode.json
{
  "$schema": "https://opencode.ai/config.json",
  "share": "disabled",
  "permissions": [
    { "action": "edit", "resource": "*", "effect": "deny" },
    { "action": "shell", "resource": "*", "effect": "ask" }
  ]
}
```

En modo Plan pedí el resumen de la aplicación: describió bien los cinco servicios (proxy, api, worker, db y cache) y el papel de nginx, pero se colaron dos invenciones (una carpeta tests/ y un «frontend React») que cazo en el ejercicio 7.

Después le pedí expresamente que modificara docker-compose.yml y quedó bloqueado:

```
Petición: «añade un servicio redis-exporter al docker-compose.yml»
El agente intenta editar el fichero…
Permission denied: edit
```

Comprobé que el fichero no se había tocado:

```
$ grep -n "exporter" docker-compose.yml
(sin resultados)
```

¿Por qué conviene un agente que no pueda escribir en la fase de análisis? Porque en esa fase solo queremos entender el sistema, no cambiarlo: con edit: deny el repositorio queda intacto (es la fuente de verdad y el punto de retorno), el agente no puede «arreglar» cosas por su cuenta mientras investiga y todas sus afirmaciones se siguen comprobando con comandos. Con bash: ask ninguna orden se ejecuta sin revisarla, y con share: disabled la sesión no puede compartirse (podría mostrar el .env).

2) Inventario de componentes. Pide al agente la lista de servicios y rellena esta tabla comprobando cada celda en los ficheros: servicio, imagen o construcción, puerto publicado al exterior, puerto interno, volúmenes, de qué servicios depende y qué hace. Indica cuántos servicios son accesibles desde fuera del equipo y por qué. (1,5p)

Respuesta:

Comprobé cada celda en los ficheros, sin fiarme de la respuesta del agente:

```
$ grep -n "ports:" docker-compose.yml
4:    ports:
$ grep -n "build:\|image:" docker-compose.yml
```

```
3:    image: nginx:1.27-alpine
14:    build: ./api
29:    build: ./worker
37:    image: postgres:16-alpine
53:    image: redis:7-alpine
$ grep -n "EXPOSE" api/Dockerfile
6:EXPOSE 3000
```

Servicio	Imagen o construcción	Puerto publicado	Puerto interno	Volúmenes	Depende de	Qué hace
proxy	nginx:1.27-alpine	8080 → 80	80	./nginx/default.conf:ro ; ./web (solo lectura)	api	sirve la web estática y hace de proxy de /api hacia la api
api	build ./api (node:20-alpine)	—	3000	—	db (con service_healthy) y cache	API REST de libros y pedidos; usa la caché y encola las facturas
worker	build ./worker (node:20-alpine)	—	— (solo llama a Redis)	—	cache	saca pedidos de la cola y «genera» la factura (hoy solo la registra en el log)
db	postgres:16-alpine	—	5432	datos-db; ./db/init.sql:ro	—	base de datos de libros y pedidos
cache	redis:7-alpine	—	6379	datos-cache	—	caché de libros (60 s) y cola cola:facturas

Solo 1 servicio es accesible desde fuera del equipo: el proxy (8080 → 80, líneas 4-5). Es el único con ports: en el compose; los demás no publican ningún puerto y solo se ven entre ellos en la red interna que crea Docker Compose, donde se localizan por el nombre del servicio (db, cache, api: líneas 18, 19, 31, y nginx/default.conf:13).

3) Recursos de la aplicación. Clasifica todos los recursos del proyecto (contenido estático, código de servidor, datos y esquema, dependencias, configuración, secretos, registros) indicando en qué fichero o

servicio vive cada uno y cuáles no deben subirse al repositorio. Comprueba si el proyecto tiene .gitignore y propón uno. (1,5p)

Respuesta:

Comprobé los recursos del proyecto y que **no tiene .gitignore**:

```
$ find . -type f | sort
./api/Dockerfile
./api/package.json
./api/src/server.js
./db/init.sql
./docker-compose.yml
./.env.example
./nginx/default.conf
./web/index.html
./worker/Dockerfile
./worker/package.json
./worker/worker.js
$ ls -a
.  ..  .env.example  api  db  docker-compose.yml  nginx  web  worker
```

Tipo de recurso	Dónde vive	¿Al repositorio?
Contenido estático	web/index.html (lo sirve el proxy, montado en solo lectura)	Sí
Código de servidor	api/src/server.js y worker/worker.js; sus Dockerfiles construyen las imágenes	Sí
Datos y esquema	Esquema y 3 libros de ejemplo: db/init.sql (solo se ejecuta la primera vez). Datos reales: volúmenes datos-db y datos-cache	init.sql sí; los volúmenes no son ficheros del repo
Dependencias	api/package.json y worker/package.json (no hay package-lock.json ni node_modules)	Sí (node_modules no)
Configuración	docker-compose.yml, nginx/default.conf y .env.example	Sí
Secretos	Valores reales de POSTGRES_PASSWORD y JWT_SECRET, que van en el .env (que no está en el repo)	Solo la plantilla .env.example; el .env nunca

Registros	No hay ficheros de log: api y worker escriben por consola (stdout de los contenedores)	Nada que subir; se ignoran *.log por si acaso

Propuse y creé este .gitignore:

```
$ cat .gitignore
# Secretos (no subir nunca)
.env

# Dependencias
node_modules/

# Registros
*.log

# Volcados de la base de datos
*.sql.gz
dump.sql
```

4) Recorrido de una petición. Pide al agente dos diagramas Mermaid: el de GET /api/libros (incluyendo la caché) y el de POST /api/pedidos (incluyendo la cola de facturas y el worker). Comprueba que el diagrama coincide con el código de api/src/server.js y worker/worker.js, corrígelo y verifica que se dibuja en el visor. (1,5p)

Respuesta:

El agente dibujó los dos diagramas, pero con dos fallos: ponía la caché en nginx con un TTL de 5 minutos y el POST lo pintaba síncrono (la respuesta 201 después de generar la factura). Los corregí contra el código:

```
$ grep -n "cache.get\|cache.set\|lPush\|status(201)" api/src/server.js
15:  const enCache = await cache.get('libros');
18:  await cache.set('libros', JSON.stringify(rows), { EX: 60 });
29:  await cache.lPush('cola:facturas', String(rows[0].id));
30:  res.status(201).json({ pedido: rows[0].id });
$ grep -n "listen\|proxy_pass" nginx/default.conf
2:    listen 80;
13:        proxy_pass http://api:3000/;
```

Verificado que se dibujan en el visor (VS Code con Markdown Preview Mermaid Support), los diagramas corregidos son estos:

Diagrama 1 — GET /api/libros (incluye la caché):

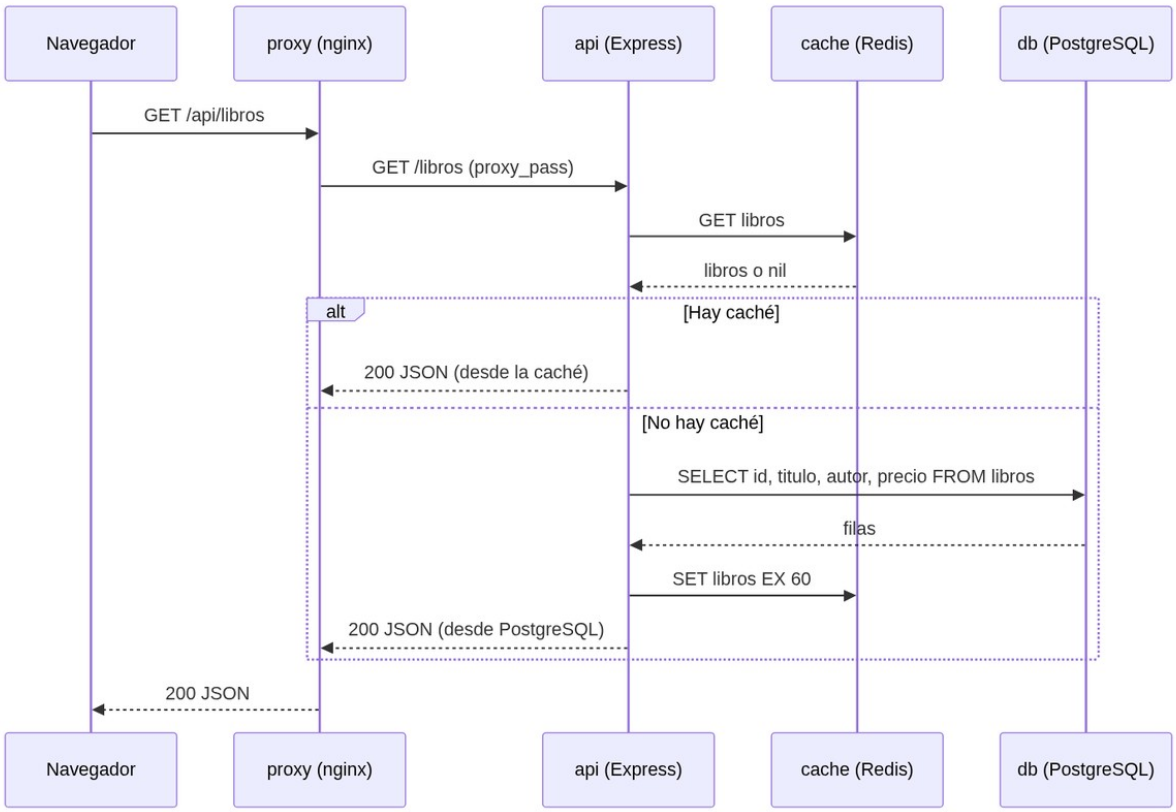
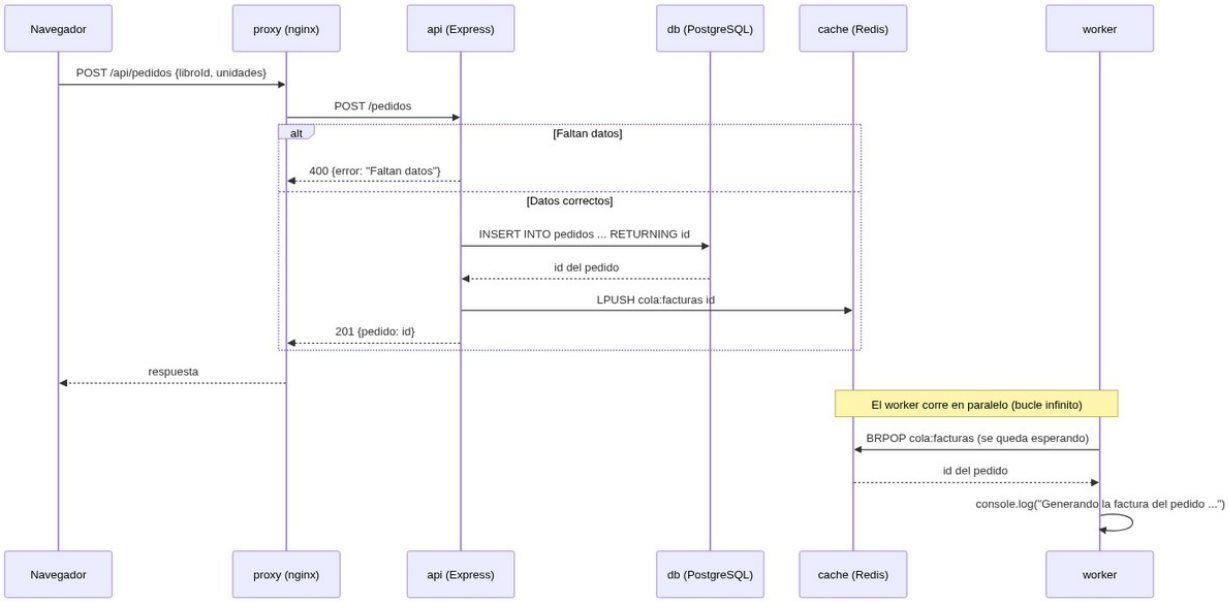


Diagrama 2 — POST /api/pedidos (cola de facturas y worker):



Detalles comprobados en el código: el proxy_pass con barra final quita el prefijo, así que a la api le llega /libros; la caché está en Redis con EX: 60 (server.js:15-18); la api encola el pedido y responde 201 sin esperar al worker (server.js:29-30); el worker hace BRPOP y de momento solo registra la «factura» en el log (worker.js:9-10).

5) Modelo de arquitectura. Pregunta al agente qué modelo de arquitectura sigue la aplicación y por qué. Contrasta su respuesta con la teoría de la unidad y con los ficheros: justifica el modelo con evidencias

concretas (número de despliegues independientes, quién habla con la base de datos, cómo se comunican los servicios) y señala qué parte de la respuesta del agente es una generalidad de manual que no se sostiene aquí. Añade ventajas e inconvenientes de este diseño para una tienda pequeña. (1,5p)

Respuesta:

El agente contestó que son «microservicios independientes, cada uno con su propia base de datos, que se comunican por REST y se orquestan con Kubernetes». Eso no se sostiene para esta aplicación.

```
$ grep -n "DATABASE_URL\|REDIS_URL" docker-compose.yml
18:      DATABASE_URL: postgres://tienda:${POSTGRES_PASSWORD}@db:5432/tienda
19:      REDIS_URL: redis://cache:6379
31:      REDIS_URL: redis://cache:6379
$ grep -n "lPush\|brPop" -r .
./api/src/server.js:29:  await cache.lPush('cola:facturas', String(rows[0].id));
./worker/worker.js:9:    const tarea = await cache.brPop('cola:facturas', 0);
```

Análisis con evidencias:

- **Despliegues independientes: ninguno.** Los 5 servicios se despliegan juntos con un solo docker compose up: 2 se construyen (api, worker) y 3 son imágenes oficiales (nginx, postgres, redis). No hay repositorios ni despliegues separados por servicio.
- **Quién habla con la base de datos: solo la api.** DATABASE_URL aparece únicamente en sus variables (línea 18). El worker no tiene credenciales ni cliente de PostgreSQL: solo REDIS_URL (línea 31), y en su código solo usa Redis.
- **Cómo se comunican:** por la red interna de Docker Compose, por nombre de servicio (db:5432, cache:6379); el único HTTP entre servicios es proxy → api. La api y el worker no hablan entre sí por REST: se coordinan por una cola en Redis (LPUSH/BRPOP sobre cola:facturas).

La generalidad de manual que no se sostiene aquí: «cada microservicio tiene su propia base de datos y se comunican por REST entre ellos» — en este proyecto hay una sola base de datos compartida, no hay API REST entre api y worker, y no hay Kubernetes por ninguna parte.

El modelo real: una aplicación pequeña desplegada como una sola unidad, con piezas de responsabilidad separada (web, api, worker, datos, caché) que comparten BD y Redis: estilo monolito distribuido.

Ventajas para una tienda pequeña: muy sencilla de desplegar (un comando), barata de alojar en una VM pequeña, con caché y cola de trabajos ya resueltas y responsabilidades bien separadas. **Inconvenientes:** la BD y Redis son puntos únicos de fallo compartidos; no hay réplicas ni alta disponibilidad; escalar obliga a duplicar contenedores sobre la misma BD; si Redis cae, se pierden a la vez la caché y la cola.

6) Ficha de requerimientos de implantación. Con ayuda del agente, redacta la ficha para poner esta aplicación en producción: hardware o instancia, software y versiones, red (dominio, DNS, puertos,

cortafuegos), seguridad y secretos, entornos, datos y copias de seguridad, monitorización y registros, fases del despliegue y plan de vuelta atrás. La ficha debe distinguir con claridad lo que la aplicación ya trae de lo que hay que añadir para producción. (2p)

Respuesta:

Redacté la ficha con ayuda del agente, pero corregí varias cosas: daba por hechas copias de seguridad y un healthcheck de la API que no existen (solo hay healthcheck de la BD, líneas 45-49):

```
$ grep -rn "backup\|pg_dump\|monitor" .
(sin resultados)
$ grep -n "healthcheck:" docker-compose.yml
45:    healthcheck:
```

Área	Ya lo trae la aplicación	Hay que añadir para producción
Hardware o instancia	Imágenes ligeras (alpine); no exige requisitos especiales	Una VM o VPS Linux (2 vCPU, 4 GB de RAM y 40 GB de disco sobran para los 5 contenedores)
Software y versiones	node:20-alpine (api y worker), nginx 1.27, postgres 16 y redis 7, definidos en los Dockerfiles y el compose	Instalar Docker Engine + plugin compose en el servidor, con versión fijada; política de actualización de imágenes
Red, puertos y cortafuegos	nginx escucha en el 80 (publicado como 8080); comunicaciones internas resueltas (api:3000, db:5432, cache:6379)	Abrir solo 80/443 en el cortafuegos y cerrar lo demás; dejar de publicar el 8080 al exterior; puertos internos solo en la red de Docker
Dominio, DNS y TLS	Nada: solo HTTP y server_name _	Dominio propio, registros DNS y certificado (p. ej. Let's Encrypt) con redirección 80 → 443 en nginx
Seguridad y secretos	Secretos por variables de entorno y plantilla .env.example; BD y caché sin puertos publicados	Crear el .env real (fuera del repo, permisos 600) con contraseñas fuertes; rotación; usar de verdad el JWT_SECRET o quitarlo (hoy no se usa); contenedores con usuario no privilegiado; actualizaciones de seguridad
Entornos	NODE_ENV=production y un único compose	Separar producción y preproducción (variables y compose distintos) y probar cada cambio antes
Datos y copias de seguridad	Volúmenes datos-db y datos-cache (los datos sobreviven a los	Copia periódica de PostgreSQL (pg_dump programado + retención) y restaurar una copia

	reinicios); init.sql crea el esquema y 3 libros solo la primera vez	de prueba; tener en cuenta que init.sql no vuelve a ejecutarse
Monitorización y registros	Ruta /salud en la API; healthcheck de la BD; logs de los contenedores por consola	Healthchecks de api, worker y cache; monitorización con avisos (p. ej. Uptime Kuma o Prometheus); centralizar y rotar los logs de Docker
Fases del despliegue	El build de las imágenes y el arranque con compose ya funcionan	Guion: VM y cortafuegos → instalar Docker → clonar el repo → crear .env → docker compose up -d --build → comprobar /salud y la web → activar TLS → copias y monitorización
Plan de vuelta atrás	El proyecto está en git y las imágenes se pueden volver a construir	Volver a la versión anterior (git checkout + docker compose up -d con la imagen previa) y restaurar el último volcado si el despliegue tocó datos; probar el rollback con antelación

7) Caza de afirmaciones falsas. A lo largo de la práctica, el agente dirá cosas que no son ciertas para esta aplicación. Recoge al menos tres, cada una con la frase del agente, el fichero y la línea que la desmienten y la afirmación correcta. (1p)

Respuesta:

Fui apuntando las afirmaciones falsas del agente y las desmentí con los ficheros:

```
$ grep -rn "jwt\|JWT\|jsonwebtoken" .
./api/package.json:13:    "jsonwebtoken": "^9.0.2"
./.env.example:3:JWT_SECRET=cambiame-tambien
./docker-compose.yml:20:      JWT_SECRET: ${JWT_SECRET}
$ find . -name "*test*"
(sin resultados)
```

Frase del agente	Fichero y línea que la desmiente	Lo correcto
«El worker genera la factura y la guarda en PostgreSQL»	worker/worker.js:9-10 (solo BRPOP y un console.log) y docker-compose.yml:31 (el worker solo recibe REDIS_URL)	Hoy no hay factura persistida: el worker solo anota en el log el pedido que saca de la cola
«La API protege los pedidos con JWT»	api/src/server.js:22-31 (POST /pedidos no comprueba ningún token; jsonwebtoken solo aparece en api/package.json:13 y el secreto en docker-compose.yml:20, sin usar)	Los endpoints son públicos: JWT está declarado pero no implementado

«El proxy sirve la tienda por HTTPS»	nginx/default.conf:2 (listen 80; no hay ssl ni 443 en todo el fichero)	Solo HTTP; el certificado hay que añadirlo (ficha, ejercicio 6)
«El proyecto trae una carpeta tests/ con pruebas»	find . -name "*test*" no devuelve nada y ls no muestra tests/	No hay ni una sola prueba en el proyecto
«PostgreSQL queda publicado al exterior en 5432»	docker-compose.yml:36-50 (el servicio db no tiene ports:; el único ports: está en el proxy, líneas 4-5)	La base de datos solo es accesible desde la red interna de Compose

Reto (ampliación)

Repetí el ejercicio 2 con dos prompts distintos: uno vago («explícame la arquitectura») y otro preciso, citando ficheros con @docker-compose.yml @api/src/server.js y diciendo qué formato quería.

	Prompt vago	**Prompt preciso**
Qué pedí	«explícame la arquitectura»	«con @docker-compose.yml y @api/src/server.js, hazme una tabla de los servicios con sus puertos y dependencias»
Qué respondió	Un texto largo y genérico: habló de microservicios y de Kubernetes, cosas que no están en el proyecto	La tabla exacta, citando líneas de los ficheros que le pasé

Conclusión: al no darle ficheros ni formato, el agente rellena con generalidades (y con errores); con contexto y formato fijado, la respuesta es comprobable y casi no hay que corregirla. El documento docs/arquitectura.md queda en el proyecto con los diagramas renderizados.

Registro de prompts

Mantén durante toda la práctica el fichero prompts.md con una tabla como esta (una fila por prompt importante):

Nº	**Prompt (resumido)**	**Qué hizo el agente**	**Qué corregí yo**
1	«Resume qué hace esta aplicación y qué ficheros la componen» (modo Plan, solo lectura)	Resumen correcto de los 5 servicios, pero se inventó una carpeta tests/ y un «frontend React»	find . -type f y ls -a: no hay tests/ ni React; web/index.html es HTML plano. Corregí el resumen
2	«Modifica docker-compose.yml para añadir un servicio de	No pudo: opencode bloqueó la edición (edit:	Nada; era justo lo que se pedía

	métricas»	deny, captura en el ejercicio 1)	comprobar
3	«Saca la tabla de inventario de servicios» (puertos, volúmenes, dependencias)	Tabla casi correcta, pero decía que el worker también escribe en PostgreSQL	grep DATABASE_URL/REDIS_URL en el compose: la BD solo la usa la api; el worker solo habla con Redis. Corregí esa fila
4	«Clasifica los recursos del proyecto y propón un .gitignore»	Clasificación correcta; proponía ignorar web/ y db/init.sql	Corregí: el estático y el esquema van al repo; fuera solo .env, node_modules, logs y volcados. Creé el .gitignore
5	«Dibuja en Mermaid el recorrido de GET /api/libros con la caché»	Bien en general, pero ponía la caché en nginx y TTL de 5 minutos	cat -n api/src/server.js: la caché es Redis con EX: 60 (líneas 15-18). Corregí el diagrama
6	«Dibuja el recorrido de POST /api/pedidos con la cola y el worker»	Lo pintó síncrono: decía que el 201 se devolvía después de generar la factura	server.js:29-30: la API encola y responde 201; el worker consume con BRPOP (worker.js:9). Corregí el orden
7	«¿Qué modelo de arquitectura sigue la aplicación y por qué?»	«Microservicios con una base de datos por servicio, comunicación REST y Kubernetes»	No hay Kubernetes, ni REST entre servicios, ni BD por servicio; evidencia: compose (5 servicios, 1 compose, 1 BD) y LPUSH/BRPOP. Reescribí con el contraste
8	«Redacta la ficha de implantación distinguiendo lo que ya trae de lo que falta»	Ficha completa, pero daba por hechas copias de seguridad y healthchecks de la API que no existen	grep healthcheck/backup: solo hay healthcheck en la BD y ninguna copia. Pasé esas filas a «hay que añadir»
9	(Reto) «explícame la arquitectura» frente a prompt preciso citando @docker-compose.yml y @api/src/server.js	El vago: respuesta genérica con cosas que no están (Kubernetes, microservicios); el preciso: ajustada al compose y al código	Me quedé con la respuesta del prompt preciso y anoté las diferencias (reto)

El registro completo está también en el fichero prompts.md del proyecto, junto a docs/arquitectura.md con el desarrollo de todos los ejercicios.